

Agentic CV Verification Report using MCP SocialGraph Tools

Name: Liu Sixiao
Student ID:1155247019

1. Problem Statement and Target Output

The goal is to build an agentic system that verifies whether a PDF CV is reliable by cross-checking the candidate's information against social-media profiles retrieved via an MCP server (SocialGraph).

For each input CV, the system outputs a single confidence score in [0, 1] (higher means more likely valid).

During evaluation, the system must process five fixed CV files (CV_1.pdf ... CV_5.pdf) and output scores = [s1, s2, s3, s4, s5] in the same order.

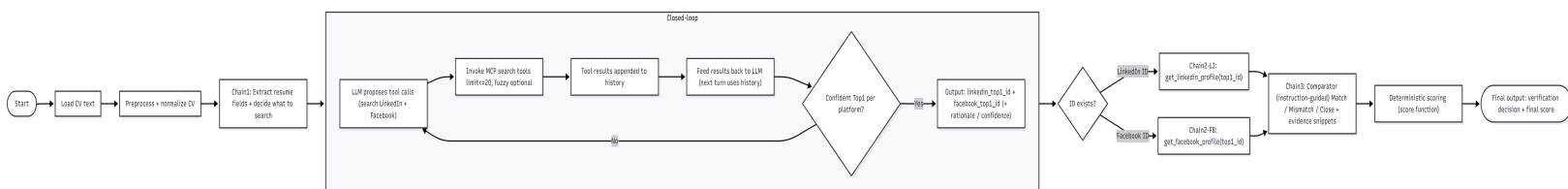
2. System Architecture and Design Decisions

My implementation is organized as a 3-chain architecture (plus deterministic scoring) to keep tool usage safe and results reproducible.

The overall design priority is: search -> select Top1 integer ID -> fetch profile -> conservative claim matching -> deterministic scoring. This separation avoids "LLM hallucination" in the final score and makes debugging easier (raw tool outputs are preserved).

- CV ingestion: convert CV PDFs into plain text, then normalize key claims (name, city/country, job, education).
- Chain1 (closed-loop Top1 selection): call MCP search tools, compute a candidate score, and pick the best candidate with a usable integer id. If confidence is low, refine the query and re-search (closed loop).
- Chain2 (evidence building): use get_*_profile function with the selected integer id to fetch full profile data and construct an evidence pack.
- Chain3 (instruction-guided classification): compare CV claims with evidence under a strict rubric, producing a structured match_report JSON.
- Deterministic score: map match_report labels into numeric scores with explicit weights and contradiction penalties, generating the final per-CV score.

2.1 Workflow Diagram (Mermaid code for the report)



3. Agent Workflow and Tool Usage Strategy

This section explains how each chain works and how MCP tools are used. The key strategy is to force “search-first” to obtain integer ids, then fetch profiles only for the selected Top1 to control tool calls and reduce false positives.

3.1 Part 2 MCP Tool Constraints (SPECIAL NOTES)

- SPECIAL NOTE: Search-first constraint: profile tools require integer ids; therefore the workflow must call search_* tools before get_*_profile.
- Type safety: get_facebook_profile only accepts integer user_id; get_linkedin_profile only accepts integer person_id. String ids like “lc...” must be rejected.
- Search result size: search tools cap limit (spec states max 20). In my code I further restrict Facebook search to limit=5 to keep selection interpretable and reduce noise.
- Closed-loop selection: because fuzzy search may return near matches, the system uses a score + margin rule; if confidence is low, it refines query terms and re-searches.

3.2 Tool Argument Normalization (type-guard)

To enforce the id-type constraints deterministically, I added a small guard that normalizes tool arguments and rejects invalid ids early. This prevents accidental misuse (e.g., passing LinkedIn ids into Facebook tools, or passing string ids into get_*_profile).

```
import re
from typing import Any, Dict, Optional

def _maybe_int(x: Any) -> Optional[int]:
    if isinstance(x, int) and not isinstance(x, bool):
        return x
    if isinstance(x, str) and x.strip().isdigit():
        return int(x.strip())
    return None

def normalize_tool_args(tool_name: str, args: Dict[str, Any]) -> Dict[str, Any]:
    if tool_name == "search_facebook_users":
        out = dict(args)
        out["limit"] = 5
        return out

    # Facebook profile int user_id
    if tool_name == "get_facebook_profile":
        uid = args.get("user_id", args.get("id"))
        uid_int = _maybe_int(uid)
        if uid_int is None:
            raise ValueError(f"get_facebook_profile requires integer user_id, got {uid!r}")
        return {"user_id": uid_int}

    # LinkedIn profile int person_id
    if tool_name == "get_linkedin_profile":
        pid = args.get("person_id", args.get("id"))
        pid_int = _maybe_int(pid)
        if pid_int is None:
            raise ValueError(f"get_linkedin_profile requires integer person_id, got {pid!r}")
        return {"person_id": pid_int}

    return args
```

3.3 Chain1: Closed-loop Candidate Search and Top1 Selection

Chain1 is responsible for

- (1) extracting a resume summary,
- (2) calling MCP search tools
- (3) returning a strict JSON object that includes raw search outputs, normalized candidates with usable integer ids, and the selected Top1 id for each platform.

A closed-loop is implemented: if the initial selection has no usable integer id or is low-confidence, the agent can refine the query (e.g., add city/country/company keywords) and run search again until it can confidently choose a single best candidate.

```
# -----  
# System/User Prompts (INLINE)# -  
# -----
```

Hard tool rules:

- Facebook:
 - Call search_facebook_users with q=<best name>, limit=5, fuzzy=true.
 - In the search result, locate the BEST integer user id field for each candidate.
 - Acceptable integer id fields (in priority order): user_id, fb_id, numeric_id, id.
 - If the candidate id is a string like "lc_..." it is NOT a usable id for profile retrieval.
- Based on the tool message being invoked, select the one best matched

- LinkedIn:
 - Call search_linkedin_people with q based on name/title/company, fuzzy=true.
 - Acceptable integer id fields (in priority order): person_id, id.
 - If the id is not an integer, it is NOT usable.

Output requirements:

- Output STRICT JSON only (no extra text).
 - You MUST include the raw search results (as returned by tools) under seed_candidates_raw so we can debug.
 - You MUST produce selection.facebook_best_user_id and selection.linkedin_best_person_id as integers whenever possible.
 - If you cannot produce a best id, set it to null AND explain why in selection.notes.
- ```
""".strip()
```

Steps:

- 1) Extract Resume information (name, location(city, country), education, experience, skills). Use null/[] for missing.
  - The name usually shows in the first line. Name should be distinguished with country and city word spelling!!!
  - Name should not include city name like "Munich, Germany"
- 2) Tool calls:
  - A) Facebook: search\_facebook\_users(q=<name>, limit=5, fuzzy=true) -ONLY consider q as name

### 3.4 Deterministic Scoring from match\_report

Instead of letting the LLM output the final numeric score directly, Chain3 returns a structured match\_report (field-level statuses + contradictions). A deterministic scoring function then converts labels into a final score. This makes grading stable and allows simple ablations (change weights/thresholds without re-prompting).

```
geo_map = {"same":1.0, "close":0.8, "unknown":0.55, "mismatch":0.0}
job_map = {"match":1.0, "strong":0.9, "partial":0.7, "unknown":0.55, "mismatch":0.0}
edu_map = {"match":1.0, "partial":0.7, "unknown":0.55, "mismatch":0.0}

name_s = (report.get("name", {}) or {}).get("status", "unknown")
city_s = (report.get("city", {}) or {}).get("status", "unknown")
country_s = (report.get("country", {}) or {}).get("status", "unknown")
job_s = (report.get("job", {}) or {}).get("status", "unknown")
edu_s = (report.get("education", {}) or {}).get("status", "unknown")
```

```

base = 0.0
base += weights["name"] * label_score(name_s, name_map)
base += weights["city"] * label_score(city_s, geo_map)
base += weights["country"] * label_score(country_s, geo_map)
base += weights["job"] * label_score(job_s, job_map)
base += weights["education"] * label_score(edu_s, edu_map)

Hard floor: name exact + (city same/close OR country same/close) => >=0.50
if name_s == "exact" and (city_s in ("same","close") or country_s in ("same","close")):
 base = max(base, 0.50)

Contradiction penalty (soft)
contradictions = report.get("contradictions") or []
n = len(contradictions)
if n >= 3:
 base *= 0.65
elif n == 2:
 base *= 0.80
elif n == 1:
 base *= 0.90

return max(0.0, min(1.0, base))
import re

def fix_name_location_glue(text: str) -> str:
 """
 Heuristic:
 - If the very first line contains letters immediately followed by a known
 'City, Country' pattern, insert a newline before the city.
 - Also normalize 'Hometown' line if present.
 """
 lines = [ln.rstrip() for ln in (text or "").splitlines()]
 if not lines:
 return text

 first = lines[0].strip()

 # pattern: "...<Name><City>, <Country>" without space between name and city
 m = re.search(r"([A-Za-z]+)([A-Z][a-z]+,\s*[A-Z][a-z]+)", first)
 if m:

```

### 3.5 Chain3: Instruction-guided

Chain3 performs conservative verification. Missing fields are treated as unknown (not mismatch). Mismatches are only declared when there is clear conflict. I also embedded an explicit rubric about city/country handling (e.g., if the CV does not disclose a current country, but social media shows a different current country, it should be classified as mismatch).

```

USER3 = """
RESUME_JSON:
{resume_json}

FACEBOOK_PROFILE_JSON:
{facebook_profile_json}

LINKEDIN_PROFILE_JSON:
{linkedin_profile_json}

EVIDENCE_JSON:
{evidence_json}

```

Return STRICT JSON with EXACTLY these top-level keys:  
name, city, country, job, education, contradictions

```

For each of name/city/country/job/education:
- it must be an object with keys: status, note
- status must be ONE of:
 - name: exact | close | mismatch | unknown
 - city/country: same | close | mismatch | unknown
 - job/education: match | strong | partial | mismatch | unknown
contradictions must be a list of strings.

```

```

Return JSON only, no extra text.
"".strip()

```

```

Build chains

p1 = ChatPromptTemplate.from_messages([("system", SYSTEM1), ("human", USER1)])
p2 = ChatPromptTemplate.from_messages([("system", SYSTEM2), ("human", USER2)])
p3 = ChatPromptTemplate.from_messages([("system", SYSTEM3), ("human", USER3)])

chain1 = p1 | llm_with_tools
chain2 = p2 | llm_with_tools
chain3 = p3 | llm

```

## 4. Sample Verification Results

The notebook includes an evaluation step with fixed groundtruth labels [1, 1, 1, 0, 0] and a decision threshold of 0.5. In the recorded run output, the system achieved perfect accuracy: it predicted decisions[1, 1, 1, 0, 0] and reported final\_score = 1.0. Because the notebook prints the final decisions (but not the raw floating scores), the final result we can report for the sample CV set is the decision vector and overall correctness.

```
{'decisions': [1, 1, 1, 0, 0], 'correct': 5, 'total': 5, 'final_score': 1.0}
```

```

▶ print(scores)# Your code should generate this list [0.2, 0.3, 0.4, 0.5, 0.6]
groundtruth = [1, 1, 1, 0, 0] # Do not modify
from langchain_core.prompts import ChatPromptTemplate
result = evaluate(scores, groundtruth)
print(result)

... [0.8800000000000002, 0.528, 0.9700000000000001, 0.27999999999999997, 0.3696]
{'decisions': [1, 1, 1, 0, 0], 'correct': 5, 'total': 5, 'final_score': 1.0}

```

Interpreting this in the required order (CV\_1 ... CV\_5): the system flags CV\_1, CV\_2, CV\_3 as valid (decision=1) and CV\_4, CV\_5 as invalid (decision=0) at threshold 0.5.

## 5. Limitations and Practical Notes

This system is optimized for the course MCP server schema and the provided tool set. In real deployments, two enhancements would be important:

- (1) more robust PDF parsing for multi-page CVs
- (2) stronger disambiguation features (e.g., company graph, mutual friends, interaction signals) when many candidates share the same name. However, the current design already emphasizes safety (type-guards), interpretability (raw tool output saved), and reproducibility (deterministic scoring).